

Project: Upgrade of AI SOC Analyst Project: Phase 4 (Active Response)

Platform: NVIDIA DGX Spark (Ubuntu 24.04 ARM64)

Analyst: Sunil Tiwari

Date: December 27, 2025

1. Executive Summary

This phase concludes the transition from passive monitoring to autonomous remediation. By integrating the Wazuh Active Response module with a custom Python remediation script, the system now identifies SSH brute-force attacks and automatically updates the host's kernel firewall (iptables) to isolate the threat. This implementation reduces the Mean Time to Remediate (MTTR) to near-zero.

2. Technical Implementation & Environment

To ensure a high-fidelity simulation, I avoided localhost whitelisting by creating a dedicated virtual network environment.

Network Architecture

- Host Interface (10.10.10.1): The "Victim" server running the Wazuh Manager and SSH service.
- Network Namespace (attacker): An isolated environment with IP 10.10.10.2, effectively simulating an external adversary.
- Remediation Script: A custom Python script (ai-block.py) that parses Wazuh JSON alerts from STDIN to execute iptables commands.

1. Integration Strategy: Custom Active Response Script

Created “**ai-block.py**” python script in path “**/var/ossec/active-response/bin/ai-block.py**”

```

● sunil77@spark-7a18:~/Desktop/projects/ai-soc-analyst$ sudo cat /var/ossec/active-response/bin/ai-block.py
#!/usr/bin/env python3
import sys
import json
import subprocess
import time
from datetime import datetime

# LOG FILE: This is where we write to Terminal 3
LOG_FILE = "/var/ossec/logs/active-responses.log"

def write_log(msg):
    with open(LOG_FILE, "a") as f:
        timestamp = datetime.now().strftime("%a %b %d %H:%M:%S %Z %Y")
        f.write(f"{timestamp} {msg}\n")

def main():
    try:
        # 1. Read the input from Wazuh (JSON format)
        input_data = sys.stdin.read()
        if not input_data:
            write_log("ai-block.py: ERROR - No input received from Wazuh")
            sys.exit(1)

        alert = json.loads(input_data)

        # 2. Extract the Source IP
        # Wazuh 4.x structure: parameters -> alert -> data -> srcip
        try:
            srcip = alert['parameters']['alert']['data']['srcip']
            command = alert['command']
        except KeyError:
            # Fallback for older rule structures
            srcip = alert.get('srcip') or "0.0.0.0"
            command = "add"

        if srcip == "0.0.0.0" or srcip == "127.0.0.1":
            write_log("ai-block.py: INFO - Skipping invalid or localhost IP")
            sys.exit(0)

        # 3. Execute the Block (ADD)
        if command == "add":
            # Check if already blocked to avoid duplicates
            check = subprocess.run(["iptables", "-C", "INPUT", "-s", srcip, "-j", "DROP"],
                                   stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)

            if check.returncode != 0:
                subprocess.run(["iptables", "-I", "INPUT", "1", "-s", srcip, "-j", "DROP"])
                write_log(f"active-response/bin/ai-block.py: add - {srcip}")
            else:
                write_log(f"active-response/bin/ai-block.py: INFO - IP {srcip} already blocked")

        # 4. Unblock (DELETE) - strictly optional for this test
        elif command == "delete":
            subprocess.run(["iptables", "-D", "INPUT", "-s", srcip, "-j", "DROP"])
            write_log(f"active-response/bin/ai-block.py: delete - {srcip}")

    except Exception as e:
        write_log(f"ai-block.py: CRITICAL ERROR - {str(e)}")

if __name__ == "__main__":
    main()

```

Image: AI Block Script

2. Permissions and Ownership

Wazuh requires strict permissions for AR scripts.

Hence, I changed Owner permissions to **root** and Group permissions to **wazuh**.

Similarly changed executables permission of the file for root to 7, Wazuh Group to 5 and none for others.

```
...
```

Bash

```
sudo chown root:wazuh /var/ossec/active-response/bin/ai-block.py
sudo chmod 750 /var/ossec/active-response/bin/ai-block.py
```

```
...
```

3. Wazuh Manager Configuration

Below is custom addition command tags to the “**ossec.conf**” file located in “**/var/ossec/etc/**”, basically modifying the Wazuh Manager Configuration to tell Wazuh:

- What script it is allowed to run
- When it should run it
- How long the response should last

Defining the command:

```
...
```

```
<command>
  <name>ai-firewall-block</name>
  <executable>ai-block.py</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>
```

```
...
```

Where, “**ai-firewall-block**” is the name of the defined command, responsible to trigger the [ai-block.py](#) script.

Next, created active response trigger based on AI Filtered Rule: 100001

```
... <active-response>
  <command>ai-firewall-block</command>
  <location>local</location>
  <rules_id>100001</rules_id>
  <timeout>600</timeout>
</active-response> ...
```

Where, “**command**” refers to the defined command to tell Wazuh which script to run. Location tag, tells wazuh to run the script within the manager itself. Rule_id tag triggers when the custom rule 100001 triggers.

Timeframe defines 600 seconds or 10 mins. Once timed out the Wazuh sends the delete command and the script removes the firewall rule.

```
GNU nano 7.2 /var/ossec/etc/ossec.conf *
</command>

<command>
  <name>win_route-null</name>
  <executable>route-null.exe</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<command>
  <name>netsh</name>
  <executable>netsh.exe</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<command>
  <name>ai-firewall-block</name>
  <executable>ai-block.py</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<active-response>
  <command>ai-firewall-block</command>
  <location>local</location>
  <rules_id>100001</rules_id>
  <timeout>600</timeout>
</active-response>

<!--
<active-response>
```

Image: Wazuh Manager Configuration

I chose to apply custom configuration to ensure the Wazuh manager maintains the state of the block (and handles the automatic timeout/unblocking).

In summary;

ARM64 Compatibility: Python 3.12 and iptables operate natively on Ubuntu 24.04 (ARM64) without architectural translation layers.

Workflow:

Rule 100001 triggers

- Wazuh Manager passes JSON to ai-block.py via STDIN
- Script executes iptables
- Manager waits 600s
- Manager passes "delete" command to script
- Script removes iptables rule.

Verification: Monitor `/var/ossec/logs/active-responses.log` to confirm execution.

TESTING- SIMULATION SETUP

Part 1: Created a Virtual Attacker - The Bridge

Following is the step taken in building a tiny, isolated “**fake attacker machine**” inside the DGX Spark host using “network namespace”, and “virtual Ethernet - veth” pair.

Created:

Host at `10.10.10.1`

An isolated namespace called `attacker` at `10.10.10.2`

A virtual cable connecting the host and attacker.

In conclusion, The Bridge setup behaves like two machines on a private LAN, but both live inside the same kernel. For this test purpose, NAT is not set - not required for the purpose of the project result.

Step 1: Created a new network namespace

```
sudo ip netns add attacker
```

This command creates a new network stack. It has its own interfaces, routes, firewall and ARP table. And is completely isolated from the host.

Step 2: Created a Virtual Ethernet Pair

```
sudo ip link add v-eth0 type veth peer name v-peer0
```

This command creates two virtual interfaces. Any data sent into one comes out of the other. This connection is like a patch cable connection.

Step 3: Moved one end to the attacker namespace

```
sudo ip link set v-peer0 netns attacker
```

This command removes `v-peer0` from the host. `v-peer0` now exists inside the attacker namespace. As a result, the host sees only `v-eth0`. Namespace sees only `v-peer0`.

Step 4: Assigned IP address to the Host side

```
sudo ip addr add 10.10.10.1/24 dev v-eth0
```

Command gives host side interface an IP (One IP) where, /24 subnet range: 10.10.10.0 - 10.10.10.255

Step 5: Activated host interface

```
`sudo ip link set v-eth0 up`
```

As interfaces are down by default, this command activates the host side of the virtual cable.

Step 6: Assigned IP address inside the attacker namespace

```
`sudo ip netns exec attacker ip addr add 10.10.10.2/24 dev v-peer0`
```

Command gives attacker namespace side interface an IP (One).

Step 7: Activated attacker namespace interface

```
`sudo ip netns exec attacker ip link set v-peer0 up`
```

Which activates the attacker's side of the virtual cable.

Network Namespace Verifications

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bash - ai-soc-analyst + - [ ] ... | x

• sunil77@spark-7a18:~/Desktop/projects/ai-soc-analyst$ sudo ip netns add attacker
[sudo] password for sunil77:
• sunil77@spark-7a18:~/Desktop/projects/ai-soc-analyst$ sudo ip link add v-eth0 type veth peer name v-peer0
• sunil77@spark-7a18:~/Desktop/projects/ai-soc-analyst$ sudo ip link set v-peer0 netns attacker
• sunil77@spark-7a18:~/Desktop/projects/ai-soc-analyst$ sudo ip addr add 10.10.10.1/24 dev v-eth0
• sunil77@spark-7a18:~/Desktop/projects/ai-soc-analyst$ sudo ip link set v-eth0 up
• sunil77@spark-7a18:~/Desktop/projects/ai-soc-analyst$ sudo ip netns exec attacker ip addr add 10.10.10.2/24 dev v-peer0
• sunil77@spark-7a18:~/Desktop/projects/ai-soc-analyst$ sudo ip netns exec attacker ip link set v-peer0 up
• sunil77@spark-7a18:~/Desktop/projects/ai-soc-analyst$ sudo ip netns list
attacker (id: 1)
• sunil77@spark-7a18:~/Desktop/projects/ai-soc-analyst$ sudo ip netns exec attacker ip addr show
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
20: v-peer0@if21: <BROADCAST,MULTICAST,UP,LOWER UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
   link/ether c2:15:f9:26:41:2c brd ff:ff:ff:ff:ff:ff link-netnsid 0
   inet 10.10.10.2/24 scope global v-peer0
       valid_lft forever preferred_lft forever
   inet6 fe80::c015:f9ff:fe26:412c/64 scope link
       valid_lft forever preferred_lft forever
• sunil77@spark-7a18:~/Desktop/projects/ai-soc-analyst$ sudo ip addr show v-eth0
21: v-eth0@if20: <BROADCAST,MULTICAST,UP,LOWER UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
   link/ether ce:61:dc:85:a1:b0 brd ff:ff:ff:ff:ff:ff link-netns attacker
   inet 10.10.10.1/24 scope global v-eth0
       valid_lft forever preferred_lft forever
   inet6 fe80::cc61:dcff:fe85:a1b0/64 scope link
       valid_lft forever preferred_lft forever
• sunil77@spark-7a18:~/Desktop/projects/ai-soc-analyst$ sudo ip netns exec attacker ping -c 3 10.10.10.1
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data:
64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=0.088 ms
64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=0.057 ms
64 bytes from 10.10.10.1: icmp_seq=3 ttl=64 time=0.057 ms

--- 10.10.10.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2056ms
rtt min/avg/max/mdev = 0.057/0.067/0.088/0.014 ms
• sunil77@spark-7a18:~/Desktop/projects/ai-soc-analyst$
```

Image: The Bridge

This screenshot shows the verification of the

- network namespace
- Interfaces inside the namespaces
- Host network namespace seeing other end of the virtual cable.
- Successful ICMP Ping test, proving the attacker can reach the host-victim across the virtual bridge.

Part 2: The Final SOC Simulation: Professional Execution

First Cleaned the old shield - **sudo iptables -D INPUT -s 10.99.99.99 -j DROP**

Step 1: Real-World attack with Hydra

Ran Hydra attack within the attacker namespace, targeting the Host virtual interface 10.10.10.1

'''

```
sudo ip netns exec attacker hydra -l root -P  
./SecLists/Passwords/Default-Credentials/ssh-betterdefaultpasslist.txt  
ssh://10.10.10.1 -t 4
```

'''

Step 2: Observe the Automated Response

```
''' sudo tail -f /var/ossec/logs/alerts/alerts.json  
'''
```

This command is used to watch OSSEC/Wazuh alerts in real time, specifically to observe whether my automated response is triggering.

When an attack or trigger happens:

- An event is detected
- A rule fires
- An alert is written to alerts.json
- Active response scripts may run
- You observe the alert appear live

Step 3: Monitoring Script ([ai-block.py](#)) execution in real-time

'''

```
sudo tail -f /var/ossec/logs/active-responses.log  
'''
```

This command is used to watch OSSEC/Wazuh automated actions as they actually execute, in real time.

```
'''
sudo watch -n 1 "iptables -nvL INPUT"
'''
```

Every second, display the incoming-traffic firewall rules, including packet and byte counters, without doing name lookups.

[illegible]

The final screenshot above concludes the project on “Active Response” as I was able to demonstrate a fully automated detection-to-remediation loop where the host now defends itself against brute-force attacks in real-time.

Below is a step-by-step breakdown of the terms and evidence found in the screenshot that confirms this completion:

Part 1: Detection: The SOC “Brain” (Terminal 2 - starting left to right)

The Rule Trigger: Multiple JSON logs can be seen recorded for Rule ID 5758.

Significance: Rule 5758 is a standard Wazuh rule for “Maximum authentication attempts exceeded”. This proves that the Wazuh Manager is accurately identifying the Hydra attack coming from the Virtual Namespace Attacker - `10.10.10.2` as a malicious brute-force event.

Part 2: Action: The AI Execution (Terminal 3)

Terminal 3, which is an active-responses.log contains the proof of the work in correlation to Terminal 1 and 2.

The Key Entry: `Fri Dec 26 22:20:27 2025 active-response/bin/ai-block.py: add - 10.10.10.2`

Significance: The term “add” signifies that the Python script ([ai-block.py](#)) received the alert from the Wazuh Manager and successfully executed the command to block the IP (10.10.10.2). The timestamp (22:20:27) matches the live attack, proving the response was instantaneous.

Part 3: The Remediation: The System Shield (Terminal 4)

Terminal 4 (far right terminal), is the firewall (iptables) shows the actual defense.

The Final Block: The command `iptables -S` shows: `-A INPUT -s 10.10.10.2/32 -j DROP`.

Significance: This is the most critical defence. The term DROP for the source IP `10.10.10.2` confirms that the kernel is now discarding all packets from the attacker. Because of this existing rule, Hydra is now completely cut off from the host system.

Part 4: The Attacker Defeat (Terminal 1)

In the far left terminal we can see Hydra has stopped making progress.

The Error: `[ERROR] could not connect to ssh://10.10.10.1:22 - Timeout connecting to 10.10.10.1`.

Significance: Term `Timeout` is the direct result of the Drop rule shown in Terminal 4, confirming that the attacker is no longer able to even reach the login screen.

Key Achievements

- **Namespace Isolation:** Proved Active Response logic across a virtual bridge to bypass standard loopback whitelists.
- **Custom Scripting:** Developed a state-compatible Python script tailored for ARM64 architecture.
- **Verified Protection:** Documented a 100% success rate in halting brute-force attempts within 15 seconds of threshold breach.